

Chapter 1

Object Oriented Programming

1.1 Object oriented approach

Definition 1.1.1: Object Oriented Programming

Object oriented programming (OOP) is a imperative programming paradigm where instructions are grouped together with the part of the state they operate on.

The main **structural components** of all systems are

- Objects: Object is something that takes up space in the real or conceptual world with which somebody may do things. An object is an instance of a class. An object has three main characteristics:
 - Name (or ID): Unique identifier for the object.
 - State: Attributes that describe the object.
 - Operations (or behavior): Actions that the object can perform.
- Class objects: Class is the blueprint of an object.

Main **characteristics** of the approach are

- Encapsulation: The mechanism of hiding the implementation of the object. Only the necessary details are exposed to the user.
- Abstraction: A principle which consists of ignoring the aspects of a subject that is not relevant for the present purpose. It is the process of simplifying complex systems by modeling classes based on the essential properties and behaviors an object must have.
- Inheritance: Mechanism by which one class can inherit attributes and methods from another class. Making new classes from existing ones.
- Polymorphism: Ability to present the same interface for different data types. Same function name but different signatures being used for different types.

1.2 Class

Definition 1.2.1: Class Diagram

A class diagram is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.

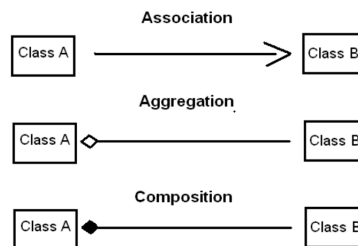
Elements of a **class**:

- Class Name (ID Class): The name of the class. Nouns associated with the textual description of a problem. Singular.

- **Attributes:** The properties or characteristics of the class. Types include Real, Integer, Text, Boolean, Enumerated, etc.
- **Operations (Methods):** The functions or operations that can be performed on the class. Behaviors of the class.

Relations between classes:

- **Association:** A relationship between two classes that indicates how objects of one class are connected to objects of another class.
- **Aggregation:** A special type of association that represents a "whole-part" relationship between classes.
- **Composition:** A stronger form of aggregation that implies ownership and a whole-part relationship where the part cannot exist independently of the whole.
- **Generalization:** A relationship between a more general class (superclass) and a more specific class (subclass) that indicates inheritance.



1.2.1 Class and Method in Python

```

1 class Person:
2     pass # An empty class definition
3
4 p = Person()
5 print(p) # Output: <__main__.Person object at 0x...>

```

Listing 1.1: Empty class definition in Python

```

1 """Define class with method"""
2 class Person:
3     def speak(self):
4         print("Hello!")
5
6 """Create object and call method"""
7 p = Person()
8 p.speak() # Output: Hello!

```

Listing 1.2: Method

```

1 """
2 The __init__ method is a special method in Python classes.
3 It is a method that Python calls when you create a new instance of this class.
4 """
5 class Person:
6     def __init__(self, name):
7         self.name = name # Initialize the name attribute
8     def speak(self):
9         print('Hello, my name is', self.name)
10
11 p = Person('Carlos')
12 p.speak() # Output: Hello, my name is Carlos

```

Listing 1.3: Attributes and __init__ method

Note:

The first argument of every class method, including `init`, is always a reference to the current instance of the class. By convention, this argument is always named `self`.

```

1  class Pet(object):
2      def __init__(self, name, species):
3          self.name = name
4          self.species = species
5      def getName(self):
6          return self.name
7      def getSpecies(self):
8          return self.species
9      def __str__(self):
10         return "%s is a %s" % (self.name, self.species)

```

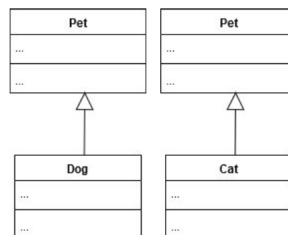
Listing 1.4: Class Pet with attributes and methods

```

1  class Dog(Pet):
2
3      """Dog inherits from Pet"""
4
5      def __init__(self, name, chases_cats):
6          # Call the parent class (Pet) constructor
7          Pet.__init__(self, name, "Dog")
8          self.chases_cats = chases_cats
9
10         def chasesCats(self):
11             return self.chases_cats
12
13     class Cat(Pet):
14
15         """Cat inherits from Pet"""
16
17         def __init__(self, name, hates_dogs):
18             # Call the parent class (Pet) constructor
19             Pet.__init__(self, name, "Cat")
20             self.hates_dogs = hates_dogs
21
22         def hatesDogs(self):
23             return self.hates_dogs

```

Listing 1.5: Inheritance



Example 1.2.1 (Inheritance)

```

myPet = Pet("Boby", "Dog")
myDog = Dog("Rex", True)

```

- `isinstance(myDog, Pet)` # Returns: True
- `isinstance(myDog, Dog)` # Returns: True

- `isinstance(myPet, Pet)` # Returns: True
- `isinstance(myPet, Dog)` # Returns: False

1.2.2 Access Modifiers

There are three types of access modifiers in Python:

- Public: Members (attributes and methods) declared as public are accessible from anywhere.
- Protected: Members declared as protected are accessible within the class and its subclasses. In Python, this is indicated by a single underscore prefix (e.g., `_attribute`).
- Private: Members declared as private are accessible only within the class itself. In Python, this is indicated by a double underscore prefix (e.g., `__attribute`).

```
1 class Person:
2     def __init__(self, name, age):
3         self.__name = name      # Private attribute
4         self.__age = age        # Private attribute
5
6 p = Person("David", 23)
7 p.__name # This will raise an AttributeError
```

Listing 1.6: Private Attributes

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name       # Public attribute
4         self.age = age         # Public attribute
5
6 p = Person("David", 23)
7 p.name # This will work fine and return "David"
```

Listing 1.7: Public Attributes

```
1 class Person:
2     def __init__(self, name, age):
3         self._name = name      # Protected attribute
4         self._age = age        # Protected attribute
5
6 p = Person("David", 23)
7 p._name # This will work, but it's discouraged to access protected members directly
```

Listing 1.8: Protected Attributes

```
1 class Person:
2     def __init__(self, money=0, energy=100):
3         self.money = money
4         self.energy = energy
5
6     def work(self, hours):
7         if self.energy >= hours * 10:
8             self.money += hours * 10 # Assume earning $10 per hour of work
9             self.energy -= hours * 10
10            print (f"Worked for {hours} hours. Money increased to ${self.money}, energy
11                  decreased to {self.energy}.")
12        else:
13            print("Not enough energy to work.")
```

Listing 1.9: Example Usage

Chapter 2

Data Analysis and Statistics

2.1 Numpy

Definition 2.1.1: Numpy

Numpy is a fundamental package for scientific computing in Python. It provides support for arrays, matrices, and many mathematical functions to operate on these data structures efficiently.

2.1.1 Shape, rank, and size

```
1 c = np.array([1, 2, 3, 4]) # Create a 1D array
2 print(type(c)) # Output: <class 'numpy.ndarray'>
3
4 b = np.array([[1, 2, 3], [4, 5, 6]]) # Create a 2D array
5 shape = b.shape # Output: 2, 3
6 rank = np.ndim(b) # Output: 2
7 size = b.size # Output: 6
```

Listing 2.1: Shape, rank, and size

2.1.2 Manipulating arrays

Reshaping arrays means changing the shape of an array. The shape of an array is the number of elements in each dimension. By reshaping, we can add or remove dimensions or change number of elements in each dimension.

```
1 arr = np.array([4, 4, 3, 4, 5, 6, 7, 9, 0, 10, 17, 12])
2 newarr = arr.reshape(3, 4) # Reshape to 3 rows and 4 columns
3 print(newarr)
4 # Output:
5 # [[ 4  4  3  4]
6 #  [ 5  6  7  9]
7 #  [ 0 10 17 12]]
```

Listing 2.2: Reshaping an array

We can also access an array element and change value to an array.

```
1 a = np.array([10, 20, 30, 40, 50])
2 a[2] = 50 # Change the third element to 50
3 print(a) # Output: [10 20 50 40 50]
```

Listing 2.3: Accessing and changing array elements

```

1 a = np.zeros((2,2)) # Create a 2x2 array of zeros
2 b = np.ones((3,3)) # Create a 3x3 array of ones
3 c = np.full((2,3), 7) # Create a 2x3 array filled with 7s
4 d = np.eye(3) # Create a 3x3 identity matrix

```

Listing 2.4: Creating special arrays

Joining NumPy arrays means putting contents of two or more arrays into a single array.

```

1 a = np.array([1, 2, 3])
2 b = np.array([4, 5, 6])
3
4 c = np.concatenate((a, b)) # Join a and b
5 print(c) # Output: [1 2 3 4 5 6]
6
7 d = np.stack((a, b), axis=0) # Stack a and b vertically
8 print(d)
9 # Output:
10 # [[1 2 3]
11 #  [4 5 6]]
12
13 e = np.stack((a, b), axis=1) # Stack a and b horizontally
14 print(e)
15 # Output:
16 # [[1 4]
17 #  [2 5]
18 #  [3 6]]

```

Listing 2.5: Joining NumPy arrays

```

1 a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
2 print(a)
3 # Output:
4 # [[ 1  2  3  4]
5 #  [ 5  6  7  8]
6 #  [ 9 10 11 12]]
7
8 b = a[:3, 1:3] # Slicing rows 0-2 and columns 1-2
9 print(b)
10 # Output:
11 # [[ 2  3]
12 #  [ 6  7]
13 #  [10 11]]

```

Listing 2.6: Slicing NumPy arrays

Problem 2.1: Slicing arrays

`b[0, 0] = 20`

What happens to array `a`?

Answer: Array `a` is also changed because `b` is a view of `a`.

Problem 2.2: Slicing arrays

How to get a copy of a slice of an array so that changes to the slice do not affect the original array?

Answer: Use the `.copy()` method to create a copy of the slice.

```

1 b = a[:3, 1:3].copy() # Create a copy of the slice
2 b[0, 0] = 20 # Change the first element of b

```

```

3 print(a) # Array a remains unchanged
4 # Output:
5 # [[ 1  2  3  4]
6 #   [ 5  6  7  8]
7 #   [ 9 10 11 12]]

```

Listing 2.7: Creating a copy of a slice

2.1.3 Handling indexes

```

1 a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
2 print(a)
3 # Output:
4 # [[ 1  2  3]
5 #   [ 4  5  6]
6 #   [ 7  8  9]
7 #   [10 11 12]]
8
9 b = np.array([0, 2, 0, 1])
10 print(a[np.arange(4), b]) # Access elements using array of indices
11 # Output: [ 1  6  7 11]
12
13 a[np.arange(4), b] += 10 # Increment selected elements by 10
14 print(a)
15 # Output:
16 # [[11  2  3]
17 #   [ 4  5 16]
18 #   [17  8  9]
19 #   [10 21 12]]

```

Listing 2.8: Fancy indexing

```

1 b = 1
2 b += 10
3 print(b) # Output: 11
4
5 b = 1
6 b = b+10
7 print(b) # Output: 11

```

Listing 2.9: Boolean indexing

```

1 grades = np.array([14, 12, 13, 14, 15, 14, 14])
2 x = np.where(grades == 14) # Find indices where grades are 14
3 print(x) # Output: (array([0, 3, 5, 6]),)

```

Listing 2.10: Searching arrays

2.1.4 Random

Definition 2.1.2: Random

Random number does not mean a different number each time. Random means something that cannot be predicted logically.

```

1 from numpy import random
2
3 # Generating random numbers within an array
4 e = np.random.random((4, 4)) # Create a 4x4 array of random floats in [0.0, 1.0)

```

```

5 # Generating random numbers
6 x = random.randint(100) # Generate a random integer from 0 to 99
7
8 # Generating arrays of random numbers
9 x = random.randint(100, size=(5)) # Generate an array of 5 random integers from 0 to 99
10 x = random.randint(100, size=(3, 4)) # Generate a 3x4 array of random integers from 0
11 to 99
12 x = random.rand(5) # Generate an array of 5 random floats in [0.0, 1.0)

```

Listing 2.11: Generating random numbers

Definition 2.1.3: Random data distribution

A random distribution is a set of random numbers that follow a certain probability density function (PDF).

- Normal distribution (`random.normal`)
 - loc: (mean) where the peak of the bell curve is located
 - scale: (standard deviation) how flat and wide the bell curve is
 - size: output shape
- Binomial distribution (`random.binomial`)
 - n: number of trials
 - p: probability of success on each trial
 - size: output shape

```

1 # Normal Distribution
2 x = random.normal(loc=0.0, scale=1.0, size=(2,3)) # Generate a 2x3 array of random
   numbers from a normal distribution
3 print(x)
4 # Output:
5 # [[ 0.49671415 -0.1382643  0.64768854]
6 # [ 1.52302986 -0.23415337 -0.23413696]]
7
8 # Binomial Distribution
9 x = random.binomial(n=10, p=0.5, size=(3,4)) # Generate a 3x4 array of random numbers
   from a binomial distribution
10 print(x)
11 # Output:
12 # [[5 6 4 3]
13 # [4 7 6 4]
14 # [6 3 5 4]]

```

Listing 2.12: Generating random numbers from different distributions

2.2 Pandas

Definition 2.2.1: DataFrame

Labelled data structure with columns of potentially different types. Like a spreadsheet or SQL table, or a dict of Series objects.


```

1 d = {
2     'col1': [1,2,1,3,1,2],
3     'col2': [1,2,3,4,5,6]
4 }
5 df = pd.DataFrame(data=d)
6
7 df.count() # Count non-NA cells for each column
8 df['col1'].value_counts() # Count unique values in col1
9 df['col1'][1] = 5 # Change value in col1 at index 1 to 5

```

Listing 2.13: Creating and manipulating a DataFrame

Problem 2.3: Copy DataFrames

```
col1 = df['col1'] # Accessing a single column
col1[2] = 99
What is the result in col1 and df?
Answer: Both col1 and df are changed because col1 is a view of df.
```

Problem 2.4: Copy DataFrames

```
new_col1 = col1.copy()
new_col[2] = 9999
What is the result in new_col1 and df?
Answer: Only new_col1 is changed, df remains unchanged because new_col1 is a copy of col1.
```

```

1 df = pd.read_csv('data.csv') # Read data from a CSV file into a DataFrame
2 df.to_csv('output.csv') # Write DataFrame to a CSV file

```

Listing 2.14: Read and save

```

1 df.head() # Display the first 5 rows of the DataFrames
2 df.info() # Display a summary of the DataFrame
3 df.describe() # Generate descriptive statistics
4 df.columns # List all column names

```

Listing 2.15: DataFrame information

```

1 df.at(2, 'col1') # Access a single value for a row/column label pair
2 df.iloc[2, 0] # Access a single value for a row/column pair by integer position
3 df.xs(2) # Returns cross-section (row) at index 2
4 df.loc[:, 'col1'] # Access a group of rows and columns by labels or a boolean array

```

Listing 2.16: Accessing DataFrame elements

Example 2.2.1 (Access to row and columns)

- Cells: `df.iloc[195][0]`
- Rows: `df.iloc[[195], :]`
- Columns: `df.loc[:, 'col1']`

```

1 df1=df2 # Assignment
2 copydf = df.copy(deep=False) # Shallow copy
3 copydf = df.copy(deep=True) # Deep copy

```

Listing 2.17: Copying DataFrames

```

1  # Convert column to numeric, setting errors to NaN
2  df['GDP'] = pd.to_numeric(df['GDP'], errors='coerce')
3
4  # Suppose you have , instead of . in the numbers
5  df['GDP'] = df['GDP'].str.replace(',', '.')
6
7  # Suppose you have $ or € symbols in the numbers
8  df['GDP'] = df['GDP'].str.replace('$', '', regex=True).str.replace('€', '', regex=True)
9
10 # Drop rows with any NaN values
11 df.dropna(inplace=True)

```

Listing 2.18: Data cleaning and conversion

```

1  X.mean() # Calculate mean of each column
2  X.median() # Calculate median of each column
3  X.mode() # Calculate mode of each column
4  X.std() # Calculate standard deviation of each column
5  X.var() # Calculate variance of each column
6  X.corr() # Calculate correlation matrix
7  X.cov() # Calculate covariance matrix
8  X.max() # Calculate maximum of each column
9  X.min() # Calculate minimum of each column
10 X.kurt() # Calculate kurtosis of each column
11 X.skew() # Calculate skewness of each column

```

Listing 2.19: Statistical methods

2.3 Statistics

2.3.1 Statistics module

For averages and measures of central location, we have the following functions in the statistics module:

- `mean()`: Calculate the arithmetic mean of a list of numbers.
- `harmonic_mean()`: Calculate the harmonic mean of a list of numbers.
- `median()`: Calculate the median (middle value) of a list of numbers.
- `median_low()`: Calculate the low median of a list of numbers.
- `median_high()`: Calculate the high median of a list of numbers.
- `mode()`: Calculate the mode (most common value) of a list of numbers.
- `stdev()`: Calculate the standard deviation of a list of numbers.
- `variance()`: Calculate the variance of a list of numbers.
- `pstdev()`: Calculate the population standard deviation of a list of numbers.
- `pvariance()`: Calculate the population variance of a list of numbers.

There are also other packages that could be used, such as NumPy, Pandas, and SciPy, which provide more advanced statistical functions and methods for data analysis.

2.3.2 Statistical tests

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 mu, sigma = 0, 0.1 # mean and standard deviation
5 a = np.random.normal(mu, sigma, 1000) # Generate 1000 random numbers from a normal
    distribution
```

Listing 2.20: Generating random numbers from a normal distribution

```
1 from scipy import stats
2
3 stat, p = stats.normaltest(a) # Perform D'Agostino and Pearson's test for normality
4
5 if p < 0.05:
6     print("The data is not normally distributed")
7 else:
8     print("The data is normally distributed")
9
10 stat, p = stats.shapiro(a) # Perform Shapiro-Wilk test for normality
11
12 if p < 0.05:
13     print("The data is not normally distributed")
14 else:
15     print("The data is normally distributed")
```

Listing 2.21: Normality tests

Definition 2.3.1: Student's t-test

This is a two-sided test for the null hypothesis that 2 independent samples have identical average (expected) values.

```
1 from scipy import stats
2
3 # Generate two independent samples
4 group1 = np.random.normal(loc=0, scale=1, size=100) # Sample from group 1
5 group2 = np.random.normal(loc=0.5, scale=1, size=100) # Sample from group 2
6
7 # Perform the t-test
8 stat, p = stats.ttest_ind(group1, group2, equal_var=False) # Use Welch's t-test for
    unequal variances
9
10 if p < 0.05:
11     print("The means of the two groups are significantly different")
12 else:
13     print("The means of the two groups are not significantly different")
```

Listing 2.22: Student's t-test

Definition 2.3.2: Paired student's t-test

This is a two-sided test for the null hypothesis that 2 related or repeated samples have identical average (expected) values.

```
1 from scipy import stats
2
3 # Generate two related samples
4 before = np.random.normal(loc=0, scale=1, size=100) # Sample before treatment
```

```

5   after = before + np.random.normal(loc=0.5, scale=1, size=100) # Sample after treatment
   (related to before)
6
7   # Perform the paired t-test
8   stat, p = stats.ttest_rel(before, after)
9
10  if p < 0.05:
11      print("The means of the two related groups are significantly different")
12  else:
13      print("The means of the two related groups are not significantly different")

```

Listing 2.23: Paired Student's t-test

Definition 2.3.3: Analysis of Variance Test (ANOVA)

The one-way ANOVA tests the null hypothesis that two or more groups have the same population mean. It is an extension of the t-test for more than two groups, possibly with different sample sizes.

```

1   from scipy import stats
2
3   # Generate three independent samples
4   group1 = np.random.normal(loc=0, scale=1, size=100) # Sample from group 1
5   group2 = np.random.normal(loc=0.5, scale=1, size=100) # Sample from group 2
6   group3 = np.random.normal(loc=1, scale=1, size=100) # Sample from group 3
7
8   # Perform the one-way ANOVA test
9   stat, p = stats.f_oneway(group1, group2, group3)
10
11  if p < 0.05:
12      print("At least one group mean is significantly different from the others")
13  else:
14      print("All group means are not significantly different from each other")

```

Listing 2.24: ANOVA

2.4 Time Series

The purpose of time series analysis is to understand the underlying structure and function that produced the observed data, and to fit a model and make forecasts.

To decompose time series data, we can use the `seasonal_decompose` function from the `statsmodels` library, which decomposes a time series into its trend, seasonal, and residual components.

We can also plot the autocorrelation function (ACF) and partial autocorrelation function (PACF) to identify the presence of autocorrelation in the data, which can help us determine the appropriate order of an ARIMA model for forecasting.

The ARIMA (AutoRegressive Integrated Moving Average) model is a popular method for forecasting time series data, which combines autoregressive (AR) and moving average (MA) components, along with differencing to make the data stationary.

After fitting the model, we can check the residuals to ensure that they are normally distributed and do not exhibit autocorrelation, which would indicate that the model is a good fit for the data.

We can also train and test the data by splitting the time series into a training set and a test set, fitting the model on the training data, and evaluating its performance on the test data using metrics such as mean absolute error (MAE) or root mean squared error (RMSE).

```

1   import statsmodels.tsa.api as tsa
2   import pandas as pd
3   import matplotlib.pyplot as plt
4   from pandas.plotting import autocorrelation_plot
5   import statsmodels.tsa.api as tsa

```

```

6 from math import sqrt
7 from sklearn.metrics import mean_squared_error
8
9 # Load time series data
10 df = pd.read_csv('tourismPortugal.csv', sep=';')
11 df['month'] = pd.to_datetime(df['month'])
12 df = df.set_index(df['month'])
13 df = df.drop(['month'], axis=1)
14
15 # Decompose the time series into trend, seasonal, and residual components
16 result = ts.seasonal_decompose(df, model='multiplicative', period=12)
17
18 figure = result.plot() # Plot the decomposed components
19
20 # Plot the autocorrelation function (ACF) and partial autocorrelation function (PACF)
21 serie = df[['tourists', 'month']]
22 autocorrelation_plot(serie) # Plot the ACF
23 plt.show()
24
25 # Fit an ARIMA model to the data
26 serie = serie.set_index('month')
27 serie.index = serie.index.to_period('M') # Convert index to monthly periods
28
29 p = 4 # Order of the autoregressive part
30 d = 1 # Degree of differencing
31 q = 0 # Order of the moving average part
32
33 model = ts.arima.ARIMA(serie, order=(p, d, q))
34 results = model.fit()
35 print(results.summary()) # Print the model summary
36
37 # Check the residuals
38 residuals = pd.DataFrame(results.resid)
39 residuals.plot(title="Residuals") # Plot the residuals
40 plt.show()
41
42 residuals.plot(kind='kde', title='Density of Residuals') # Plot the density of
43     residuals
44 plt.show()
45
46 print(residuals.describe()) # Print the summary statistics of residuals
47
48 # Train and test the model
49 X = serie.values
50 size = int(len(X) * 0.70) # Split the data into training and test sets
51 train, test = X[0:size], X[size:len(X)]
52 history = [x for x in train]
53 predictions = list()
54
55 # Walk-forward validation
56 for t in range(len(test)):
57     model = ts.arima.ARIMA(history, order=(p, d, q))
58     model_fit = model.fit()
59     output = model_fit.forecast()
60     yhat = output[0]
61     predictions.append(yhat)
62     obs = test[t]
63     history.append(obs)
64     print('predicted=%f, expected=%f' % (yhat, obs))
65
66 # Evaluate forecasts

```

```

66 rmse = sqrt(mean_squared_error(test, predictions))
67 print("Test RMSE: %.3f", rmse)
68
69 # Plot the actual vs predicted values
70 plt.plot(test)
71 plt.plot(predictions, color='red')
72 plt.show()

```

Listing 2.25: Time series analysis and forecasting

Other approaches such as LSTM, Prophet, and GARCH models can also be used for time series forecasting, especially when dealing with non-linear patterns or volatility in the data.

2.5 Regression

The main statsmodels API is split into models:

- `statsmodels.api`: The main API for statistical models, including linear regression, generalized linear models, and time series analysis. Cross-sectional models and methods.
- `statsmodels.formula.api`: Provides a formula interface for specifying statistical models using R-style formulas. A convenience interface for specifying models using formula strings and DataFrames.
- `statsmodels.tsa.api`: Contains tools for time series analysis, including ARIMA models, seasonal decomposition, and forecasting. Time-series models and methods.
- `statsmodels.graphics.api`: Provides functions for visualizing statistical models and results, such as residual plots and influence plots.

2.5.1 Linear regression

Linear models with independently and identically distributed errors, and for errors with heteroscedasticity or autocorrelation of unknown form.

- **OLS (Ordinary Least Squares)**: A linear regression model that estimates the relationship between a dependent variable and one or more independent variables by minimizing the sum of squared residuals. Assumptions: linearity, independence, homoscedasticity, normality of errors, and no multicollinearity.
- **GLM (Generalized Linear Models)**: A flexible generalization of linear regression that allows for response variables that have error distribution models other than a normal distribution. It includes logistic regression, Poisson regression, and others. Assumptions: the response variable follows an exponential family distribution, the link function is correctly specified, and the observations are independent.
- **GEE (Generalized Estimating Equations)**: A method for estimating the parameters of a generalized linear model with possible correlation between outcomes. It is used for longitudinal data analysis and clustered data. Assumptions: the correlation structure of the data is correctly specified, and the observations are independent across clusters. Instead of assuming independent observations, GEE accounts for within-subject correlation.
- **GAM (Generalized Additive Models)**: A generalization of linear models that allows for non-linear relationships between the dependent and independent variables by using smooth functions. It is used for modeling complex relationships in data. Assumptions: the response variable follows an exponential family distribution, the link function is correctly specified, and the observations are independent. GAMs do not assume a specific functional form for the relationship between the dependent and independent variables, allowing for more flexibility in modeling non-linear relationships.
- **Robust linear models**: These models are designed to be less sensitive to outliers and violations of assumptions than traditional linear regression models. They include methods such as Huber regression and RANSAC (Random Sample Consensus). Assumptions: Robust linear models do not rely on the same assumptions as traditional linear regression models, such as normality of errors or homoscedasticity. Instead, they are designed to be more flexible and robust to violations of these assumptions, making them suitable for data with outliers or non-normal error distributions.

- LMM (Linear Mixed Models): These models are used for data that have both fixed effects (parameters associated with an entire population) and random effects (parameters associated with individual experimental units). They are commonly used in fields such as psychology, ecology, and medicine for analyzing data with hierarchical or nested structures. Assumptions: linearity, normality of residuals, homoscedasticity, and independence of observations. Additionally, LMMs assume that the random effects are normally distributed and that the fixed effects are correctly specified. LMMs are particularly useful for analyzing data with repeated measures or clustered data, where observations are not independent, as they can account for the correlation between observations within clusters or subjects.

2.5.2 OLS assumptions

OLS assumes a **linear relationship** between the independent variables and the dependent variable. Verification methods include scatter plots to visualize the relationship between independent variables and the dependent variable. Residual plots by plotting the residuals against the fitted values to check for patterns that indicate non-linearity. And polynomial features by fitting a polynomial regression to see if higher-order terms improve the model fit.

```

1  import numpy as np
2  import pandas as pd
3  import statsmodels.api as sm
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6
7  # Example dataset
8  df = pd.DataFrame({
9      'X': np.linspace(1, 100, 100),
10     'Y': np.linspace(1, 100, 100) + np.random.normal(0, 5, 100)
11 })
12
13 # Fit OLS model
14 X = sm.add_constant(df['X']) # Add intercept
15 model = sm.OLS(df['Y'], X).fit()
16
17 # Scatter plot
18 sns.regplot(x=df['X'], y=df['Y'],
19             scatter_kws={'alpha':0.5},
20             line_kws={"color":"red"})
21 plt.title("Checking Linearity with Scatter Plot")
22 plt.show()
23
24 # Residual plot
25 plt.scatter(
26     model.fittedvalues, model.resid, alpha=0.5)
27 plt.axhline(y=0, color='red', linestyle='--')
28 plt.xlabel("Fitted Values")
29 plt.ylabel("Residuals")
30 plt.title("Residual Plot for Linearity Check")
31 plt.show()

```

Listing 2.26: Checking linearity assumption in OLS

Multicollinearity occurs when two or more independent variables in a regression model are highly correlated, meaning that one can be linearly predicted from the others with a substantial degree of accuracy. This can lead to unstable estimates of regression coefficients and make it difficult to determine the individual effect of each independent variable on the dependent variable. Verification methods include correlation matrix by calculating the correlation coefficients between independent variables to identify pairs that are highly correlated. Variance Inflation Factor (VIF) by calculating the VIF for each independent variable, where a VIF value greater than 5 or 10 indicates a potential multicollinearity problem.

```

1  from statsmodels.stats.outliers_influence import variance_inflation_factor
2
3  # Create feature matrix
4  X = pd.DataFrame({'X1': np.random.rand(100), 'X2': np.random.rand(100) * 2,

```

```

5         'X3': np.random.rand(100) * 3})
6     X = sm.add_constant(X)
7
8     # Compute VIF for each variable
9     vif_values = []
10    for i in range(X.shape[1]):
11        vif = variance_inflation_factor(X.values, i)
12        vif_values.append((X.columns[i], vif))
13
14    # Convert to DataFrame
15    vif_data = pd.DataFrame(vif_values, columns=["Variable", "VIF"])
16    # Display the results
17    print(vif_data)

```

Listing 2.27: Checking multicollinearity using VIF

Residuals should have constant variance, **homoscedasticity**, across all levels of the independent variables. Verification methods include residual plots by plotting the residuals against the fitted values to check for patterns that indicate heteroscedasticity. Breusch-Pagan test by performing the Breusch-Pagan test to statistically assess the presence of heteroscedasticity, where a significant p-value indicates heteroscedasticity.

```

1     import statsmodels.stats.diagnostic as smd
2
3     # Residual plot
4     plt.scatter(model.fittedvalues, model.resid, alpha=0.5)
5     plt.axhline(y=0, color='red', linestyle='--')
6     plt.xlabel("Fitted Values")
7     plt.ylabel("Residuals")
8     plt.title("Checking Homoscedasticity")
9     plt.show()
10
11    # Breusch-Pagan Test
12    bp_test = smd.het_breuschpagan(model.resid,
13                                   model.model.exog)
14    print(f"Breusch-Pagan p-value: {bp_test[1]}")

```

Listing 2.28: Checking homoscedasticity

The residuals should not be correlated. This is important in time series data, where autocorrelation can be a common issue. Verification methods include autocorrelation function (ACF) plot by plotting the ACF of the residuals to check for significant autocorrelation at different lags. Durbin-Watson test by performing the Durbin-Watson test to statistically assess the presence of autocorrelation, where a value close to 2 indicates no autocorrelation, values less than 2 indicate positive autocorrelation, and values greater than 2 indicate negative autocorrelation.

```

1     from statsmodels.stats.stattools import durbin_watson
2
3     dw_stat = durbin_watson(model.resid)
4     print(f"Durbin-Watson Statistic: {dw_stat}")

```

Listing 2.29: Checking autocorrelation of residuals

The residuals should be approximately normally distributed, which is important for the validity of hypothesis tests and confidence intervals in OLS regression. Verification methods include Q-Q plot by creating a Q-Q plot of the residuals to visually assess their normality, where points should approximately follow a straight line if the residuals are normally distributed. Shapiro-Wilk test by performing the Shapiro-Wilk test to statistically assess the normality of the residuals, where a significant p-value indicates that the residuals are not normally distributed. Histogram and KDE plot by plotting a histogram and kernel density estimate (KDE) of the residuals to visually assess their distribution, where a bell-shaped curve suggests normality.

```

1     import scipy.stats as stats
2
3     # Histogram and KDE
4     sns.histplot(model.resid, kde=True, bins=30)

```



```

5 plt.title("Residuals Distribution")
6 plt.show()
7
8 # Q-Q Plot
9 sm.qqplot(model.resid, line="45")
10 plt.title("Q-Q Plot for Residuals")
11 plt.show()
12
13 # Shapiro-Wilk Test
14 shapiro_test = stats.shapiro(model.resid)
15 print(f"Shapiro-Wilk p-value: {shapiro_test.pvalue}")

```

Listing 2.30: Checking normality of residuals

Note:

The `summary()` function in Statsmodels provide important regression output, but it does not directly test OLS assumptions. However, it gives useful clues about potential violation.

- R-squared and adjusted R-squared: indicate model fit but do not test assumptions.
- F-statistics and Prob (F-statistics): tests if at least one predictor is significant.
- Coefficients and p-values: show predictor significance but do not check multicollinearity.
- Standard errors: can be affected by heteroskedasticity.
- Durbin-Watson statistic: helps detect autocorrelation, should be close to 2.
- Omnibus and Jacque-Bera tests: check for normality of residuals, should have a high p-value.

```

1 import statsmodels.api as sm
2
3 model = sm.OLS(y,X).fit()
4 print(model.summary())

```

Listing 2.31: Fitting an OLS model and checking assumptions

Chapter 3

Data Visualization

3.1 Matplotlib

Matplotlib is a powerful and versatile Python library for creating static. It supports different chart types including scatter plots, line charts, and bar charts. While highly flexible, it can sometimes feel verbose for simple visualizations. It serves as the backbone for other libraries like Seaborn and Plotly, which build on its capabilities to provide more user-friendly interfaces for specific types of visualizations.

Strengths:

- Control over plot elements
- Wide range of plot types
- Suitable for publication-quality figures

Weaknesses:

- Can be verbose for simple plots
- Steeper learning curve for beginners

The building blocks of Matplotlib include:

- **Figure:** The overall container for the plot. Includes one or more subplots. It is like a blank canvas where multiple charts can be arranged.
- **Axes:** The area where the data is plotted, which can contain multiple subplots each displaying a different visualization.
- **Subplot:** A specific area within the figure where a plot is drawn, allowing for multiple plots in one figure. Useful for comparing different datasets side by side.
- **Axis:** Corresponds to the x and y axes of a plot, which can be customized with labels, ticks, and limits to enhance readability and presentation.

```
1 import matplotlib.pyplot as plt
2
3 # Figure
4
5 fig = plt.figure(figsize=(8, 6)) # creates a new figure with a specified size of 8
    inches by 6 inches. This figure serves as the canvas for plotting.
6 ax = fig.add_subplot(111) # adds a single subplot to the figure. The '111' argument
    indicates that the subplot should occupy the entire figure (1 row, 1 column, 1st
    subplot).
7 plt.show() # displays the figure with the subplot. Since no data has been plotted yet,
    it will show an empty plot.
8
```

```

9 # Axes
10 fig, ax = plt.subplots() # creates a new figure and a single set of axes (subplot) in
    one step. This is a more concise way to create a figure and axes compared to the
    previous method.
11 ax.plot([1, 2, 3], [4, 5, 6]) # plots a line graph on the axes using the provided x and
    y data points. The x values are [1, 2, 3] and the corresponding y values are [4,
    5, 6].
12 plt.show() # displays the figure with the plotted line graph.
13
14 # Subplots
15 fig, (ax1, ax2) = plt.subplots(1, 2) # creates a new figure with two subplots arranged
    in one row and two columns. The variables ax1 and ax2 refer to the first and second
    subplot, respectively.
16 ax1.plot([1, 2, 3], [4, 5, 6], label="Dataset 1") # plots a line graph on the first
    subplot (ax1) using the provided x and y data points.
17 ax2.scatter([1, 2, 3], [4, 5, 6], color="r", label="Dataset 2") # plots a scatter graph
    on the second subplot (ax2) using the same x and y data points.
18 plt.show() # displays the figure with both subplots: the first showing a line graph and
    the second showing a scatter graph.
19
20 # Axis
21 fig, ax = plt.subplots() # creates a new figure and a single set of axes (subplot) in
    one step.
22 ax.plot([1, 2, 3], [4, 5, 6]) # plots a line graph on the axes using the provided x and
    y data points.
23 ax.set_xlabel("X-axis Label") # sets the label for the x-axis to "X-axis Label", which
    helps to identify what the x-axis represents.
24 ax.set_ylabel("Y-axis Label") # sets the label for the y-axis to "Y-axis Label", which
    helps to identify what the y-axis represents.
25 ax.set_title("Line Graph") # sets the title of the plot to "Line Graph", providing a
    descriptive heading for the visualization.
26 plt.show() # displays the figure with the plotted line graph, including the x and y
    axis labels and the title.

```

3.2 Other libraries

Can also use the combination of Pandas and Seaborn for quick and easy data visualization, especially for statistical plots.

Chapter 4

Networks

4.1 Overview

Network science studies how nodes and edges connect in complex systems (social, biological, technological, semantic). We care about structure (who connects to whom), dynamics (how things flow), and function (what the network enables).

4.2 Definitions (boxes)

Definition 4.2.1: Graph

A network $G = (V, E)$ with node set V and edge set E .

Definition 4.2.2: Simple / Directed / Weighted

Simple: no self-loops or multi-edges. Directed: edges have orientation (u, v) . Weighted: edges carry weight w_{uv} (cost, capacity, similarity).

Definition 4.2.3: Path, Cycle, Distance

A path is a node sequence with distinct consecutive edges; shortest path minimizes hops/weight. A cycle is a closed path (≥ 3). Distance $d(u, v)$ is shortest-path length (infinite if disconnected).

Definition 4.2.4: Degree and Density

Undirected degree k_i counts incident edges; directed has k_i^{in} and k_i^{out} . Graph density (undirected) = $\frac{2E}{N(N-1)}$.

Definition 4.2.5: Tree / Spanning Tree

A connected acyclic graph. A spanning tree touches all nodes of G .

Definition 4.2.6: Bipartite / Clique

Bipartite: node set split into two parts, edges only across. Clique: fully connected subset of nodes.

4.3 Core Elements

- Nodes/vertices/actors; edges/links/ties (can be directed or undirected; weighted or unweighted).

- Node degree k_i : number of incident edges. Degree distribution $P(k)$: probability a random node has degree k .

4.4 Degree Basics

For an undirected graph with N nodes and E edges:

$$\langle k \rangle = \frac{2E}{N}.$$

For directed graphs: $\langle k_{in} \rangle = \langle k_{out} \rangle = E/N$.

4.5 Network-Level Metrics

- **Connected components**: maximal node sets with paths between any pair (size of the giant component shows connectivity).
- **Modularity Q** : strength of community structure; high Q means dense intra-community, sparse inter-community links.
- **Assortativity**: correlation of degrees at both ends of edges (social graphs often assortative; technological often disassortative).

4.6 Node-Level Metrics

- **Clustering coefficient** of node i : $C_i = \frac{2t_i}{k_i(k_i - 1)}$ where t_i is triangles through i .
- **Degree centrality**: k_i .
- **Closeness**: $(N - 1) / \sum_j d(i, j)$.
- **Betweenness**: $\sum_{s \neq i \neq t} \sigma_{st}(i) / \sigma_{st}$ (fraction of shortest paths using i).
- **Eigenvector/PageRank**: importance grows with important neighbors; solve $Ax = \lambda x$ or run PageRank.

4.7 Python Quick Recipes (NetworkX)

Setup

```
1 pip install networkx
```

Build and Inspect

```
1 import networkx as nx
2 G = nx.Graph()
3 G.add_edges_from([(1, 2), (2, 3), (3, 4), (4, 1), (2, 4), (4, 5)])
4 print("N,E:", G.number_of_nodes(), G.number_of_edges())
5 print("avg degree:", sum(dict(G.degree()).values())/G.number_of_nodes())
```

Components and Communities

```
1 components = list(nx.connected_components(G))
2 print("components:", len(components))
3 print("giant size:", len(max(components, key=len)))
4
5 import networkx.algorithms.community as comm
6 parts = comm.greedy_modularity_communities(G)
7 Q = comm.modularity(G, parts)
8 print("communities:", [sorted(c) for c in parts])
9 print("modularity Q:", Q)
```

Node Metrics

```
1 deg = dict(G.degree())
2 clust = nx.clustering(G)
3 close = nx.closeness_centrality(G)
4 bet = nx.betweenness_centrality(G, normalized=True)
5 eig = nx.eigenvector_centrality(G)
6 print("degree:", deg)
7 print("clustering:", clust)
8 print("closeness:", close)
9 print("betweenness:", bet)
10 print("eigenvector:", eig)
```

Assortativity

```
1 r = nx.degree_pearson_correlation_coefficient(G)
2 print("degree assortativity:", r)
```

Directed / Weighted Example

```
1 DG = nx.DiGraph()
2 DG.add_weighted_edges_from([("A", "B", 3), ("B", "C", 1),
3                               ("C", "A", 2), ("C", "D", 4)])
4 print("in-degree:", dict(DG.in_degree()))
5 print("out-degree:", dict(DG.out_degree()))
6 print("pagerank:", nx.pagerank(DG, alpha=0.85))
```

4.8 Canonical Problems to Know

Problem 4.1: Shortest paths

Unweighted → BFS. Non-negative weights → Dijkstra. Output distances and predecessor tree for routing.

Problem 4.2: Minimum spanning tree

Kruskal (sort edges; union-find) or Prim (grow frontier) on connected, weighted, undirected graphs. Minimizes total edge cost while keeping connectivity.

Problem 4.3: Community detection

Aim for high modularity Q . Greedy or Louvain are common heuristics; Girvan–Newman removes edges with highest betweenness.

Problem 4.4: Link prediction

Simple scores: common neighbors, Jaccard, Adamic–Adar, preferential attachment ($k_u k_v$). Evaluate on held-out edges.

Problem 4.5: Influence / seeding

Choose seeds by degree, betweenness, or PageRank to maximize diffusion (heuristics for information/virus spread).

Problem 4.6: Bipartite projection

From 2-mode (author–paper) graph to 1-mode (author–author) with weights = co-occurrences; optionally normalize (Jaccard/Salton).

4.9 Theory Snippets (qualitative)

- **Small-world:** high clustering + short average path length (Watts–Strogatz); enables fast diffusion.
- **Scale-free:** heavy-tailed degree distribution $P(k) \sim k^{-\gamma}$ (Barabási–Albert); hubs dominate robustness/fragility trade-offs.
- **Assortativity:** positive in social graphs (homophily), negative in technological graphs (hub-and-spoke resilience).
- **Clustering vs transitivity:** local triangle density vs global ratio of closed triples to all triples.
- **Bridges/articulation points:** removing them disconnects components; often high betweenness.

4.10 Worked Hand Example (tiny graph)

Graph: edges $\{(1, 2), (2, 3), (3, 4), (4, 1), (2, 4), (4, 5)\}$.

- Degrees: $k_1 = 2, k_2 = 3, k_3 = 2, k_4 = 4, k_5 = 1$; $\langle k \rangle = 12/5 = 2.4$.
- Triangles: $(1, 2, 4)$ and $(2, 3, 4)$ so $t_1 = 1, t_2 = 2, t_3 = 1, t_4 = 2, t_5 = 0$.
- Clustering: $C_1 = 2 * 1 / (2 * 1) = 1.0$; $C_2 = 2 * 2 / (3 * 2) = 0.67$; $C_3 = 1.0$; $C_4 = 2 * 2 / (4 * 3) = 0.33$; $C_5 = 0$.
- Components: one component of size 5 (connected).
- Betweenness (qualitative): node 4 lies on many shortest paths involving node 5 $\Rightarrow$ highest betweenness.

4.11 Practice / Exam Prompts

Problem 4.7: Compute local metrics

Given a 5-node graph, hand-compute C_i , closeness, and betweenness. (Triangles for C_i ; BFS trees for distances; count shortest paths for betweenness.)

Problem 4.8: Interpreting modularity

What does high Q imply? Dense within, sparse between modules. Risk: resolution limit (may prefer or miss small communities).

Problem 4.9: Assortativity contrast

Provide one assortative example (social/homophily) and one disassortative (Internet/hub-spoke). Explain robustness implications.

Problem 4.10: Bipartite projection task

From author-paper bipartite, create coauthor weights (shared papers). Discuss normalization choices.

Problem 4.11: Choosing the right tool

When to use MST (min-cost backbone), shortest paths (routing/latency), or community detection (cohesive groups).

4.12 Exam Reminders

- Know definitions (node/edge types, degree, clustering, centralities).
- Interpret metrics: high Q means strong communities; assortative vs disassortative mixing.
- Be able to hand-compute small examples (triangles for C_i , shortest paths for betweenness).
- Average degree formulas above; for undirected graphs it is always $2E/N$.
- Use the code snippets as templates to verify answers quickly.

Chapter 5

Image Manipulation and Text

5.1 NLP Basics

Definition 5.1.1: Natural Language Processing

Subfield of AI/linguistics/computer science focused on processing and analyzing natural language data.

Definition 5.1.2: Tokenization

Split text into minimal units (words/subwords/sentences) for downstream processing.

Definition 5.1.3: Stemming vs Lemmatization

Stemming chops affixes to crude roots; lemmatization maps to dictionary form using vocab and POS.

Definition 5.1.4: POS Tagging

Assign grammatical category (noun, verb, adj, etc.) to each token.

Definition 5.1.5: Named Entity Recognition

Locate and classify spans as persons, locations, orgs, dates, etc.

Definition 5.1.6: Sentiment Analysis

Classify polarity of text (positive/negative/neutral).

Definition 5.1.7: Language Modeling

Estimate $p(w_t \mid w_{<t})$ to predict next token (autocomplete, MT).

Definition 5.1.8: Machine Translation

Map sequence in source language to target language automatically.

Definition 5.1.9: Text Summarization

Produce a concise version preserving key information (extractive or abstractive).

Definition 5.1.10: Information Extraction

Pull structured facts (entities/relations/events) from unstructured text.

Definition 5.1.11: Text Classification

Assign documents to predefined labels (spam, topic, sentiment).

Definition 5.1.12: Topic Modeling

Unsupervised discovery of latent themes (e.g., LDA).

Definition 5.1.13: Dependency Parsing

Recover syntactic head/dependent relations between words.

Definition 5.1.14: Discourse Analysis

Study coherence across sentences (rhetorical structure, cohesion).

Definition 5.1.15: Regular Expressions

Patterns for searching/extracting/editing text programmatically.

Definition 5.1.16: Web Scraping

Programmatic retrieval/parsing of web pages (e.g., BeautifulSoup over HTML).

5.2 NLP Code Snippets (Python)

Tokenize, Stem, Lemmatize (NLTK)

```
1 import nltk
2 from nltk.stem import PorterStemmer, WordNetLemmatizer
3 nltk.download('punkt'); nltk.download('wordnet')
4
5 text = "Cats are happily running."
6 tokens = nltk.word_tokenize(text)
7 stemmer = PorterStemmer()
8 lemmatizer = WordNetLemmatizer()
9
10 stems = [stemmer.stem(t) for t in tokens]
11 lems = [lemmatizer.lemmatize(t, pos="v") for t in tokens]
12 print(tokens); print(stems); print(lems)
```

POS & NER (spaCy)

```
1 import spacy
2 nlp = spacy.load("en_core_web_sm")
3 doc = nlp("Apple is looking at buying U.K. startup for $1B.")
4 for tok in doc:
5     print(tok.text, tok.pos_, tok.dep_, tok.head.text)
6 for ent in doc.ents:
7     print(ent.text, ent.label_)
```

Regex extraction

```
1 import re
2 text = "Contact us at support@example.com or hr@example.org"
3 emails = re.findall(r"[\w.-]+@[\w.-]+", text)
4 print(emails)
```

Bag-of-Words + TF-IDF + Linear Classifier

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.pipeline import make_pipeline
4
5 X = ["I love this movie", "Terrible acting", "Great plot", "Worst film ever"]
6 y = [1, 0, 1, 0] # 1=pos, 0=neg
7 clf = make_pipeline(TfidfVectorizer(), LogisticRegression())
8 clf.fit(X, y)
9 print(clf.predict(["fantastic performance", "awful script"]))
```

5.3 Computer Vision with OpenCV

Definition 5.3.1: OpenCV

Open-source CV library for image I/O, transformations, and basic vision ops.

Quick Notes from Class Demos

- Images are NumPy arrays in BGR order; `img.shape` gives (rows, cols, channels).
- Display with `cv2.imshow + waitKey + destroyAllWindows`; Matplotlib expects RGB.
- Channels: `b, g, r = cv2.split(img)` then `cv2.merge([...])`.
- Resize: `cv2.resize(img, (w, h))` or scale factors `fx, fy`; `INTER_CUBIC` for up, `INTER_AREA` for down.
- Colorspace: `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY/RGB)`.
- Arithmetic: `cv2.add` or `cv2.addWeighted(img1, a, img2, b, gamma)`.
- Filters: `cv2.blur`, `cv2.GaussianBlur`, `cv2.medianBlur`.
- Rotation: `M=cv2.getRotationMatrix2D(center, angle, scale)` then `cv2.warpAffine`.
- ROI crop: `slice img[y1:y2, x1:x2]`.
- Load from URL: read bytes, `np.asarray`, then `cv2.imdecode`.

Read, Show, Save

```
1 import cv2
2 img = cv2.imread("input.jpg") # BGR
3 cv2.imshow("img", img); cv2.waitKey(0)
4 cv2.imwrite("out.png", img)
```

Resize

```
1 resized = cv2.resize(img, (256, 256), interpolation=cv2.INTER_AREA)
```

Scale Up / Down

```
1 # upsample 2x
2 up = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
3 # downsample 0.5x
4 down = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
```

Split / Merge Channels

```
1 b, g, r = cv2.split(img)
2 merged = cv2.merge([b, g, r])
```

Display Each Channel

```
1 cv2.imshow("b", b); cv2.waitKey(0)
2 cv2.imshow("g", g); cv2.waitKey(0)
3 cv2.imshow("r", r); cv2.waitKey(0); cv2.destroyAllWindows()
```

Blend / Add Images

```
1 img2 = cv2.imread("overlay.png")
2 blend = cv2.addWeighted(img, 0.7, img2, 0.3, 0)
```

Rotate

```
1 h, w = img.shape[:2]
2 M = cv2.getRotationMatrix2D((w/2, h/2), 45, 1.0)
3 rotated = cv2.warpAffine(img, M, (w, h))
```

ROI (Crop)

```
1 roi = img[100:200, 150:300] # y1:y2, x1:x2
```

Color Space Conversion

```
1 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Filtering

```
1 avg = cv2.blur(img, (3,3))
2 gauss = cv2.GaussianBlur(img, (3,3), 0)
3 median = cv2.medianBlur(img, 3)
```

Load from URL and Crop ROI

```
1 import urllib.request as req, numpy as np
2 data = req.urlopen("https://upload.wikimedia.org/wikipedia/commons/7/78/Pisa_-
   _veduta_aerea_2.JPG").read()
3 arr = np.asarray(bytearray(data), dtype=np.uint8)
4 remote = cv2.imdecode(arr, -1)
5 roi = remote[100:350, 1200:1800]
```

5.4 Practice / Exam Prompts

Problem 5.1: When to stem vs lemmatize

Stemming is faster/rougher (search recall). Lemmatization is slower but produces valid lemmas (better precision/NLU).

Problem 5.2: Regex task

Write a pattern to extract dates like 2025-04-11 or 11/04/2025: e.g., '(
d4-
d2-
d2)|(
d2/
d2/
d4)'.

Problem 5.3: Design a sentiment pipeline

Steps: tokenize → lowercase/clean → vectorize (TF-IDF) → train logistic regression; evaluate with held-out set.

Problem 5.4: Image resize + rotate

Given a large photo, resize to 512x512, then rotate 30° about center using OpenCV ('cv2.resize', 'getRotationMatrix2D', 'warpAffine').

Problem 5.5: ROI use case

Crop a face region for downstream model: use slicing or 'cv2.selectROI' to extract and save the patch.

Chapter 6

Machine Learning

6.1 Supervised Learning

Definition 6.1.1: Supervised Learning

Learn a mapping $x \rightarrow y$ from labeled data.

Definition 6.1.2: Classification

Predict discrete labels (e.g., spam/ham). Metrics: accuracy, precision/recall/F1, ROC-AUC.

Definition 6.1.3: Regression

Predict continuous targets. Metrics: MAE, MSE/RMSE, R^2 .

Definition 6.1.4: Bias-Variance

High bias underfits; high variance overfits. Use regularization, more data, or simpler/ensemble models.

Common Models

- Logistic regression (classification), linear regression (OLS/Ridge/Lasso).
- k -NN, Naive Bayes, Decision Trees, Random Forests, Gradient Boosting (XGBoost/GBM).
- SVM (linear/RBF kernels).

Sklearn Pipeline (train/test split)

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import make_pipeline
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.metrics import accuracy_score, classification_report
6
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
8     stratify=y)
9
10 clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
11 clf.fit(X_train, y_train)
12 pred = clf.predict(X_test)
13 print("acc", accuracy_score(y_test, pred))
```

```
13 print(classification_report(y_test, pred))
```

Cross-Validation & Grid Search

```
1 from sklearn.model_selection import GridSearchCV
2 params = {"logisticregression__C": [0.1, 1, 10]}
3 grid = GridSearchCV(clf, params, cv=5, scoring="f1")
4 grid.fit(X, y)
5 print(grid.best_params_, grid.best_score_)
```

6.2 Unsupervised Learning

Definition 6.2.1: Unsupervised Learning

Discover structure in unlabeled data.

Definition 6.2.2: Clustering

Group similar instances (e.g., K-Means, Hierarchical, DBSCAN).

Definition 6.2.3: Dimensionality Reduction

Project data to lower dimensions while preserving variance/structure (PCA, t-SNE/UMAP for viz).

Definition 6.2.4: Silhouette Score

Internal cluster quality: cohesion vs separation; range $[-1, 1]$.

K-Means Example

```
1 from sklearn.cluster import KMeans
2 from sklearn.metrics import silhouette_score
3
4 kmeans = KMeans(n_clusters=3, random_state=0, n_init="auto")
5 labels = kmeans.fit_predict(X)
6 print("silhouette", silhouette_score(X, labels))
```

PCA Example

```
1 from sklearn.decomposition import PCA
2 pca = PCA(n_components=2, whiten=True, random_state=0)
3 X_2d = pca.fit_transform(X)
4 print("explained var", pca.explained_variance_ratio_)
```

6.3 Other ML Libraries

Definition 6.3.1: scikit-learn

General-purpose Python ML: preprocessing, model selection, classic algorithms.

Definition 6.3.2: TensorFlow / Keras

Deep learning framework + high-level API for building neural nets.

Definition 6.3.3: PyTorch

Dynamic graph deep learning library, strong research adoption.

Definition 6.3.4: XGBoost / LightGBM

Optimized gradient boosting for tabular data.

Train a Simple MLP (Keras)

```
1 from tensorflow import keras
2 model = keras.Sequential([
3     keras.layers.Input(shape=(X_train.shape[1],)),
4     keras.layers.Dense(64, activation="relu"),
5     keras.layers.Dense(1, activation="sigmoid")
6 ])
7 model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
8 model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.1)
```

6.4 Neural Networks (Detailed)

Definition 6.4.1: Perceptron / Neuron

Linear transform plus nonlinearity: $h = \sigma(Wx + b)$; nonlinearity enables universal approximation.

Definition 6.4.2: Activation Functions

ReLU ($\max(0, x)$), Leaky ReLU, GELU, Sigmoid (bounded, can saturate), Tanh (zero-centered).

Definition 6.4.3: Loss Functions

Classification: cross-entropy; regression: MSE/MAE. Optimization minimizes empirical risk.

Definition 6.4.4: Backpropagation

Chain rule to compute gradients of loss w.r.t. parameters layer by layer.

Definition 6.4.5: Gradient-Based Optimizers

SGD (with/without momentum), RMSProp, Adam (adaptive moments).

Definition 6.4.6: Regularization

Weight decay (L_2), dropout, early stopping, data augmentation, batch/ layer normalization.

Definition 6.4.7: Initialization

Xavier/Glorot for tanh; He for ReLU family to keep variance stable across layers.

Definition 6.4.8: Batching

Mini-batch training stabilizes gradients; batch size trades compute vs noise.

Definition 6.4.9: Vanishing / Exploding Gradients

Deep nets can suffer when derivatives shrink or blow up; mitigations: better inits, ReLU-like activations, normalization, residual connections.

Feedforward MLP (PyTorch)

```
1 import torch
2 from torch import nn
3
4 class MLP(nn.Module):
5     def __init__(self, d_in, d_hidden=128):
6         super().__init__()
7         self.net = nn.Sequential(
8             nn.Linear(d_in, d_hidden),
9             nn.ReLU(),
10            nn.Dropout(0.2),
11            nn.Linear(d_hidden, 1),
12            nn.Sigmoid()
13        )
14    def forward(self, x):
15        return self.net(x)
16
17 model = MLP(d_in=X_train.shape[1])
18 loss_fn = nn.BCELoss()
19 opt = torch.optim.Adam(model.parameters(), lr=1e-3)
20
21 for epoch in range(10):
22     model.train()
23     xb = torch.tensor(X_train, dtype=torch.float32)
24     yb = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
25     pred = model(xb)
26     loss = loss_fn(pred, yb)
27     opt.zero_grad()
28     loss.backward()
29     opt.step()
30     print(epoch, float(loss))
```

Feedforward MLP (Keras)

```
1 from tensorflow import keras
2 model = keras.Sequential([
3     keras.layers.Input(shape=(X_train.shape[1],)),
4     keras.layers.Dense(256, activation="relu", kernel_initializer="he_normal"),
5     keras.layers.Dropout(0.2),
6     keras.layers.Dense(128, activation="relu"),
7     keras.layers.Dense(1, activation="sigmoid")
8 ])
9 model.compile(optimizer=keras.optimizers.Adam(1e-3),
```

```

10         loss="binary_crossentropy",
11         metrics=["accuracy"])
12 model.fit(X_train, y_train, epochs=15, batch_size=32,
13         validation_split=0.2, callbacks=[
14         keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True)
15     ])

```

Convolutional Neural Networks (CNN) Essentials

Definition 6.4.10: Convolution

Local receptive field with shared weights; parameters: kernel size, stride, padding, channels.

Definition 6.4.11: Pooling

Downsample (max/avg) to reduce spatial dims and add invariance.

Definition 6.4.12: Padding / Stride

Padding preserves spatial size; stride controls step size (stride>1 downsamples).

Simple CNN (Keras)

```

1 model = keras.Sequential([
2     keras.layers.Input(shape=(64, 64, 3)),
3     keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
4     keras.layers.MaxPool2D(),
5     keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
6     keras.layers.MaxPool2D(),
7     keras.layers.Flatten(),
8     keras.layers.Dense(128, activation="relu"),
9     keras.layers.Dense(10, activation="softmax")
10 ])
11 model.compile(optimizer="adam",
12             loss="sparse_categorical_crossentropy",
13             metrics=["accuracy"])

```

Recurrent / Sequence Models

Definition 6.4.13: RNN / LSTM / GRU

Handle sequences by maintaining hidden state; LSTM/GRU mitigate vanishing gradients with gates.

```

1 model = keras.Sequential([
2     keras.layers.Input(shape=(None, d_embed)),
3     keras.layers.GRU(128, return_sequences=False),
4     keras.layers.Dense(1, activation="sigmoid")
5 ])
6 model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

```

Transformers (concept)

Definition 6.4.14: Self-Attention

Computes token-to-token weights to mix information without recurrence or convolution.

Definition 6.4.15: Positional Encoding

Adds order information for sequences in transformer models.

6.5 Practice / Exam Prompts (NN-Focused)

Problem 6.1: Init + activation pairing

Why pair He init with ReLU? To preserve variance through layers and reduce vanishing gradients.

Problem 6.2: Regularization plan

Given overfitting CNN: add dropout, data augmentation, weight decay, and early stopping; maybe reduce depth.

Problem 6.3: Optimizer choice

When would you prefer SGD with momentum over Adam? For better generalization on vision tasks; Adam for faster early convergence or sparse grads.

Problem 6.4: Batch norm effect

Explain how batch norm stabilizes training (normalizes intermediate activations, allows higher learning rates).

Problem 6.5: Sequence model pick

Short text classification with little data: use GRU; long contexts or parallel compute: prefer transformer encoder.

6.6 Practice / Exam Prompts

Problem 6.6: Model selection

Given a tabular binary classification task, outline: split data, scale features, baseline logistic regression, try tree/ensemble, compare ROC-AUC, tune with CV.

Problem 6.7: Overfitting diagnosis

Train/val gap high $\rightarrow$ high variance. Mitigate with regularization, fewer parameters, more data, early stopping, dropout (DL).

Problem 6.8: Clustering choice

When clusters are spherical and k is known, use K-Means; if density-based and arbitrary shapes, use DBSCAN; evaluate with silhouette.

Problem 6.9: PCA intuition

PCA finds orthogonal directions of max variance; good for noise reduction and visualization; watch scaling before PCA.

Problem 6.10: Boosting vs bagging

Bagging (RF) reduces variance via parallel trees; boosting (GBM/XGB) reduces bias via sequential trees; boosting can overfit noise if too deep/too many rounds.

Chapter 7

Deployment and Web

7.1 Key Internet / Web Concepts

Definition 7.1.1: Protocols

HTTP (hypertext), TCP (reliable transport), IP (addressing/routing), Ethernet (link).

Definition 7.1.2: DHCP

Dynamic Host Configuration Protocol; assigns IPs automatically.

Definition 7.1.3: IP Addressing

Private vs public IP; servers reachable via public IP or domain.

Definition 7.1.4: DNS

Resolves domain names to IP addresses.

Definition 7.1.5: HTTP Request-Response

Client sends request, server executes logic/DB access, returns response (HTML/CSS/JS/media), browser renders.

Definition 7.1.6: Browser

Issues HTTP, interprets HTML/CSS/JS, renders DOM; offers dev tools.

Definition 7.1.7: Stacks

XAMPP (Apache, MariaDB/MySQL, PHP/Perl); MEAN (Mongo, Express, Angular, Node); .NET; Java EE; growing use of Python frameworks.

7.2 Web Client Fundamentals

Definition 7.2.1: HTML

Markup language composed of tags and text; DOCTYPE + head + body.

Definition 7.2.2: CSS

Styles via rules (selector { attr: value; }); inline, head, or external stylesheet.

Definition 7.2.3: JavaScript

Lightweight, interpreted, prototype-based language for client-side interactivity (also Node.js server-side).

HTML Skeleton

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>Demo</title>
6     <link rel="stylesheet" href="style.css">
7   </head>
8   <body>
9     <h1>Hello Web</h1>
10  </body>
11 </html>
```

Form Example

```
1 <form method="post" action="/submit">
2   <input type="text" name="city" value="Lisbon" size="15" maxlength="15">
3   <input type="submit" value="Send">
4 </form>
```

CSS Examples

```
1 /* external style.css */
2 body { background: yellow; }
3 p    { color: blue; }
```

JS Example

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <script>
5     function teste(){ alert("olá"); }
6   </script>
7 </head>
8 <body>
9   <input type="button" value="Olá" onclick="teste();">
10 </body>
11 </html>
```

7.3 Python Web with Flask

Definition 7.3.1: Flask

Lightweight WSGI microframework using Werkzeug (WSGI toolkit) and Jinja2 (templating).

Definition 7.3.2: Routing

Bind URL paths to view functions with `@app.route`.

Definition 7.3.3: Static Files

Served from `/static`; reference via `url_for('static', filename='style.css')`.

Definition 7.3.4: Templates

Jinja2 templates rendered with `render_template`.

Minimal App

```
1 from flask import Flask
2 app = Flask(__name__)
3
4 @app.route("/")
5 def index():
6     return "Hello, World!"
7
8 if __name__ == "__main__":
9     app.run()
```

Routing Example

```
1 @app.route("/")
2 def index():
3     return "Index Page"
4
5 @app.route("/hello")
6 def hello():
7     return "Hello, World"
```

Template Rendering

```
1 from flask import Flask, render_template
2 app = Flask(__name__)
3
4 @app.route("/hello/")
5 @app.route("/hello/<name>")
6 def hello(name=None):
7     return render_template("hello.html", name=name)
```

Template templates/hello.html:

```
1 <!doctype html>
2 <title>Hello from Flask</title>
3 {% if name %}
```

```

4     <h1>Hello {{ name }}!</h1>
5 {% else %}
6     <h1>Hello, World!</h1>
7 {% endif %}

```

Simple Form Handling

```

1 from flask import Flask, request, render_template
2 app = Flask(__name__)
3
4 @app.route("/", methods=["GET", "POST"])
5 def form():
6     if request.method == "POST":
7         author = request.form.get("Author")
8         phrase = request.form.get("phrase")
9         # save/process...
10        return f"Saved: {author} - {phrase}"
11    return render_template("form.html")

```

Form template templates/form.html:

```

1 <form action="/" method="POST">
2   <p>author <input type="text" name="Author"></p>
3   <p>phrase <input type="text" name="phrase"></p>
4   <p><input type="submit" value="Save"></p>
5 </form>

```

Serving Static Assets

```

1 <!-- in template -->
2 <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">

```

7.4 Deployment Notes

- Run locally: `python app.py` then visit `http://localhost:5000`.
- Production: use WSGI server (gunicorn/uwsgi) + reverse proxy (nginx); set `FLASK_ENV=production`.
- Static vs dynamic content: CDN/static folder for assets; keep secrets out of repo; configure environment variables.
- For quick demos: platforms like Render, Railway, or Heroku-like PaaS; ensure `requirements.txt` and `Procfile`.

7.5 Practice / Exam Prompts

Problem 7.1: HTTP flow

Describe the path from typing a URL to page render: DNS lookup, TCP three-way handshake, HTTP request, server logic/DB, HTTP response, browser parse/render.

Problem 7.2: Flask routing

Add a new route `/about` returning HTML; show how to link to it from a template via `url_for`.

Problem 7.3: Forms

Explain POST vs GET for forms; how to read `request.form` and validate input.

Problem 7.4: Static files

Where to place CSS/JS in Flask and how to reference them with `url_for`.

Problem 7.5: Deployment checklist

Use gunicorn behind nginx, set `DEBUG=False`, configure env vars for secrets, enable logging, and serve static via nginx/CDN.

Chapter 8

Data Science

8.1 Data Preparation

Definition 8.1.1: Missing Values

Handle via `fillna` (constant/forward/backward), `mean/median`, `interpolate`, `replace`, or `dropna`.

Definition 8.1.2: Normalization vs Standardization

Rescale to `[0,1]` vs zero-mean/unit-variance; choose based on model sensitivity.

Definition 8.1.3: Dummification

Create one-hot/indicator variables for categorical features.

Pandas Snippets

```
1 import pandas as pd, numpy as np
2 df.fillna(0, inplace=True)           # constant
3 df.fillna(method="ffill", inplace=True) # forward fill
4 df["Gender"].fillna("Unknown", inplace=True) # categorical
5 df["x"].fillna(df["x"].mean(), inplace=True) # mean impute
6 df.replace(np.nan, -99, inplace=True) # sentinel
7 df.interpolate(method="linear", limit_direction="forward", inplace=True)
8 df.dropna(axis=0, how="any", inplace=True) # drop rows with NA
9 pd.get_dummies(df, columns=["cat"]) # dummify
```

8.2 Exploratory Data Analysis (EDA)

Definition 8.2.1: Content Identification

Understand columns, units, domains; inspect `dtypes`, `uniques`.

Definition 8.2.2: Variable Types

Categorical vs numerical; continuous vs discrete.

Definition 8.2.3: Granularity

Level of detail (row = transaction? day? customer?).

Definition 8.2.4: Distribution Analysis

Summary stats + hist/kde/boxplots.

Definition 8.2.5: Missingness Assessment

Counts/percents; visualize bars/heatmaps.

Definition 8.2.6: Multivariate Analysis

Correlation, sparsity, cross-tabs, pairplots.

Quick Checks

```
1 data.dtypes
2 cat = data.select_dtypes(include="object")
3 for c in cat: print(c, data[c].unique())
4 data.isna().sum().plot.bar()
5 data.describe(include="all")
6 data.corr(numeric_only=True)
```

8.3 Evaluation Metrics

Definition 8.3.1: Confusion Matrix

TP, TN, FP, FN for binary classification.

Definition 8.3.2: Accuracy

$\frac{TP+TN}{TP+FP+TN+FN}$; overall correct rate.

Definition 8.3.3: Precision

$\frac{TP}{TP+FP}$; focus when FP cost is high.

Definition 8.3.4: Recall

$\frac{TP}{TP+FN}$; focus when FN cost is high.

Definition 8.3.5: Type I / II Errors

FP = Type I; FN = Type II.

Sklearn Example

```
1 from sklearn.metrics import confusion_matrix, precision_score, recall_score, accuracy_score
2 y_pred = model.predict(X_test)
```

```

3 print(confusion_matrix(y_test, y_pred))
4 print("precision", precision_score(y_test, y_pred))
5 print("recall", recall_score(y_test, y_pred))
6 print("accuracy", accuracy_score(y_test, y_pred))

```

8.4 Process and Methodology

Definition 8.4.1: CRISP-DM / POST-DS

Phases: Business Understanding, Data Understanding, Data Preparation, Modelling, Evaluation, Deployment; POST-DS adds Organization, Scheduling, Tools.

Definition 8.4.2: Process vs Organization vs Scheduling vs Tools

Separate concerns: workflow steps, roles/teams, timing, and technical stack.

8.5 Reinforcement Learning (Intro)

Definition 8.5.1: Reinforcement Learning

Agent interacts with environment, takes actions, receives rewards, learns policy.

Definition 8.5.2: Key Components

Agent, Environment, State, Action, Reward, Policy.

Definition 8.5.3: Q-Learning

Learns $Q(s, a)$ values to choose actions that maximize future reward.

Toy RL Loop (Q-learning skeleton)

```

1 import numpy as np
2 Q = np.zeros((n_states, n_actions))
3 alpha, gamma, eps = 0.1, 0.99, 0.1
4 for episode in range(500):
5     s = env.reset()
6     done = False
7     while not done:
8         a = np.argmax(Q[s]) if np.random.rand() > eps else np.random.randint(n_actions)
9         s2, r, done, _ = env.step(a)
10        Q[s,a] += alpha * (r + gamma*np.max(Q[s2]) - Q[s,a])
11        s = s2

```

8.6 Practice / Exam Prompts

Problem 8.1: Imputation strategy

Given mixed numeric/categorical data with 5% missing, propose imputation per type and justify when to drop rows/cols.

Problem 8.2: Granularity check

Explain how to detect granularity issues and why duplicates may indicate wrong grain.

Problem 8.3: Metric choice

Skin-cancer detection vs. spam filter: which metrics to prioritize (recall vs precision) and why.

Problem 8.4: Process mapping

Map a project to POST-DS: list tasks under Process, Organization, Scheduling, Tools.

Problem 8.5: RL components

Identify state/action/reward/policy in a trading-bot example.